ITA17: NLP Final Project
Neural Machine Translation with Transformers

Improving JP-> EN Transformer

Applying this model to Sinhala->EN Translation and Improving

TJ_Translate

m5291087: Withanaarachchi Tithira

m5291021: Hayafuji Fumiaki

Improving JP-> EN Transformer

1. Problems Identified in the Baseline Model -> Introduced Solutions(1)

Tokenization Pipeline was Incorrect.

Incorrect application:

MeCab-Owakati tokenization -> english
NLTK word_tokenize -> Japanese

```
src: <UNK>
tgt: How much time does the kid have left ?
out: I ' m sorry , I ' m not a little one.

これは本です。
src: これは本です。
out: I ' m sorry , I ' m not a little one.
```

Correction:

Mecab-Owakati tokenization -> Japanese.
NLTK word_tokenize -> English

```
これは本です。
src: これ は 本 です 。
out: This is French is .
```

1. Problems Identified in the Baseline Model -> Introduced Solutions(2)

Wrong Data Ordering

Incorrect application:

src->English
tgt -> Japanese

```
src: <UNK>
tgt: How much time does the kid have left ?
out: I ' m sorry , I ' m not a little one.

これは本です。
src: これは本です。
out: I ' m sorry , I ' m not a little one.
```

Correction:

src -> Japanese.
tgt -> English

```
これは本です。
src: これ は 本 です 。
out: This is French is .
```

1. Problems Identified in the Baseline Model -> Introduced Solutions(3)

No model loading / checkpointing

Added the below code:

```
train_dataloader = DataLoader(train_X, train_Y, batch_size)
valid_dataloader = DataLoader(valid_X, valid_Y, batch_size, shuffle=  False)

# Check if a checkpoint already exists
if os.path.exists(ckpt_path):
    print("Checkpoint found. Loading model and skipping training."  )

    # For inference / evaluation, DataParallel is not necessary
    model = Transformer(**model_args).to(device)
    ----------------------------
    ----------------------------
    TRAIN_FLAG =  False

else:
    print("No checkpoint found. Training from scratch."  )
    model = Transformer(**model_args).to(device)
    ---------------------------------------
    ----------------------------------------
    TRAIN_FLAG =  True
```

2. Other Improvements.

- **Increase the data set size(from 25000 -> 100000).**
- **Introducing Beam Search and Beam Search+Length Penalty.**
- **Increasing the number of epochs(20 -> 30).**
- **Adding Visualizations related to Analysis and Attention(Heat maps, Performance Graphs).**

3. Beam Search Decoding.

Why do we need it?

- Greedy decoding picks **only the highest-probability word** at each step.
- But the best final translation may require choosing a word that is **not** the top choice at some early step.
- Beam Search keeps **multiple best partial translations** at once → reducing mistakes.

Core Idea

Instead of keeping **1** best continuation (greedy), Beam Search keeps **K** best candidates (beam size = K):

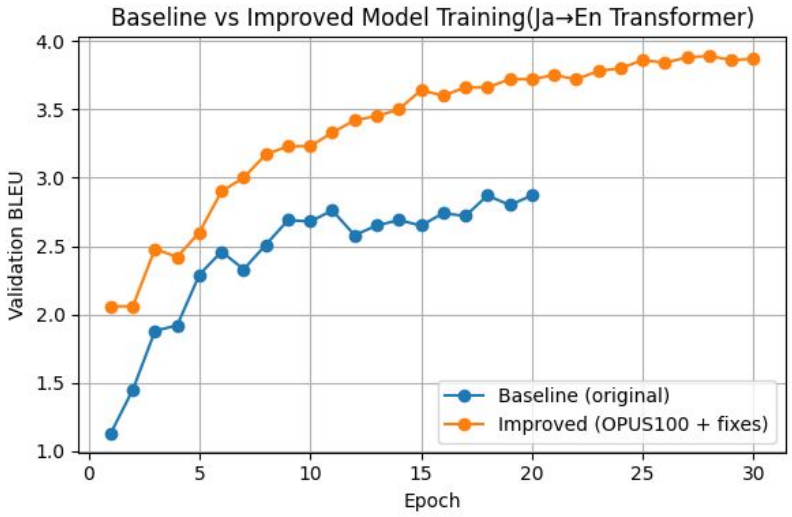
- At each time step:
  1. Expand every partial sentence with **all possible next words**.
  2. Compute their probabilities.
  3. Keep only the **top K** most promising ones.
- Continue until EOS token appears or max length is reached.

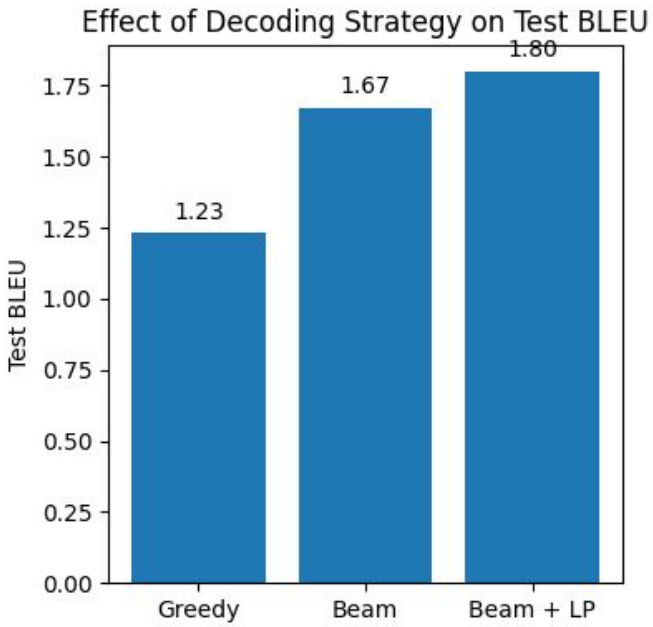
Optional: Length Penalty

- Prevents beam search from preferring short sentences.
- Helps generate complete, grammatically correct translations.

$$LP(L) = \left(\frac{5+L}{6}\right)^{\alpha} \qquad \text{score} = \frac{\text{logprob}}{LP(L)}$$

4. BLEU score improvements.





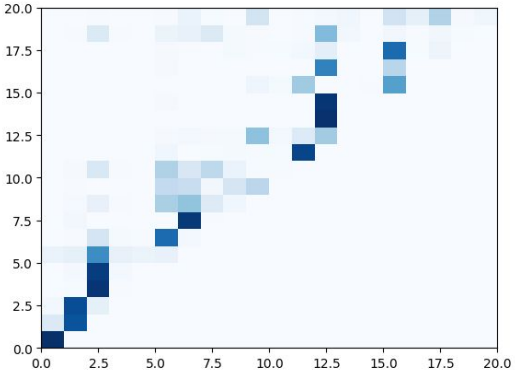
5. Comparison of Translations.

Baseline Model(From LMS)		
Japanese Input (src)	**Expected Meaning**	**Baseline Output (model from the LMS)**
<UNK>	Unknown token, but should produce something minimal	I'm sorry, I'm not a little one.
これは本です。	"This is a book."	I'm sorry, I'm not a little one.

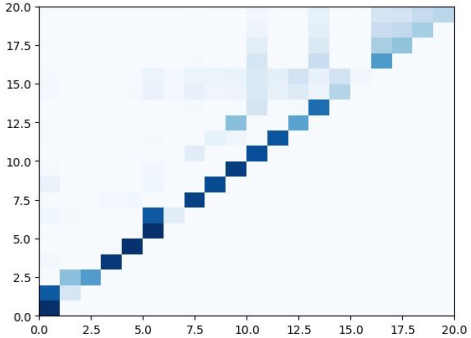
Improved Model			
Japanese Input	**Greedy**	**Beam**	**Beam + LP**
これは本です。	This is French is	This is French .	This is French .
もし そう なっ たら	If they 're talking Crawford .	Well , that 's why we 're talking .	Well , that 's why we 're talking .
ほんとうにありがとうございます。	Thank you very much .	Thank you very much .	Thank you very much .

6. Attention Heat Maps

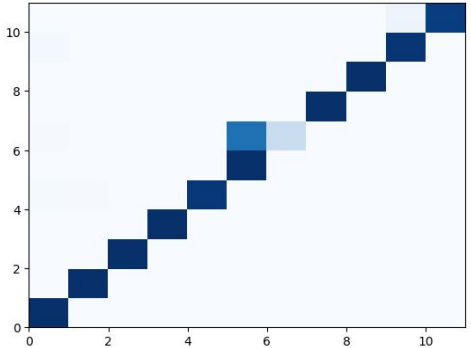
Baseline Model(Layer 1:Middle layer)



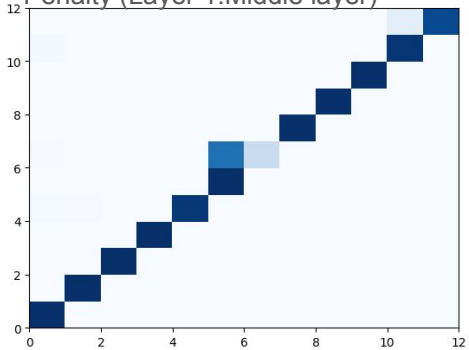
Improved Model with Greedy Decoding(Layer 1:Middle layer)



Improved Model with Beam Search Decoding(Layer 1:Middle layer)



Improved Model with Beam Search Decoding + Length Penalty (Layer 1:Middle layer)



7. Future Works(JP->EN model)

- Improve the data accuracy while manually auditing the translations(Data contains inaccurate translations as well).

- OPUS100 dataset has 1 million data altogether. If the resources allow, training using all the data is recommended.

- Can increase the number of epochs after increasing the dataset size.

Applying improved model to SN->EN and Efficiency Improving

1. Challenges for Transformer-based translation task

Require substantial computational resources and memory

⇨ **difficult to deploy on limited capacity device**

It need effective compression techniques to reduce computational time & memory usage

⇨ **Dynamic Sparse Training**

2. Core Idea & Algorithm of Dynamic Sparse Training

Step 1 — Initialize Weights and Sparse Mask

Step 2 — Update Mask During Training (Prune & Grow)

- **Prune:** Remove connections with the smallest weight magnitudes (set mask = 0).
- **Grow:** Activate new connections where gradient magnitudes are large (set mask = 1).
- Newly activated connections are initialized with small random weights.

Step 3 — Repeat the Prune-and-Grow Cycle

- Continue this process throughout training.
- The model maintains the same overall sparsity, while the connection structure
- dynamically evolves to find a better topology.

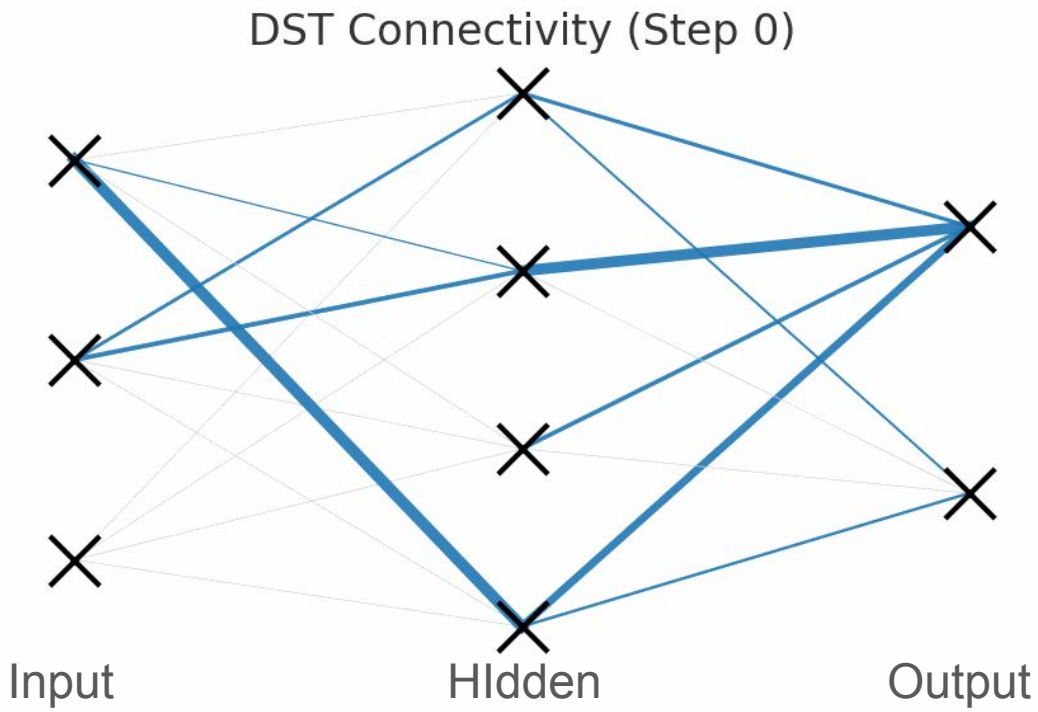
Algorithm 1 Dynamic Sparse Training (Pseudo-code)

Require: Weights θ, mask M (same shape as θ), sparsity S, total steps T, update interval Δt, prune ratio p, data $\mathcal{D}$

Ensure: Sparse trained model (θ, M)

```
Initialization
Initialize dense weights θ
Initialize binary mask M so that density = 1 − S (e.g., random sparse mask)
Apply initial sparsity: θ ← θ ⊙ M
for t = 1 to T do
    (x, y) ← next mini-batch from D
    ŷ ← f(x; θ ⊙ M)
    L ← ℒ(ŷ, y)
    g ← ∇_θ L
    θ ← optimizer_step(θ, g, M)                    ▷ Ignore entries where M = 0
    if t mod Δt = 0 then                           ▷ Prune-and-grow step
        for each layer ℓ do
            n_active ← ‖M_ℓ‖_0
            k ← ⌊p · n_active⌋                     ▷ Number of weights to reallocate
            /* Prune: smallest |θ| among active weights */
            P_ℓ ← indices of the k smallest |θ_ℓ| where M_ℓ = 1
            M_ℓ[P_ℓ] ← 0,   θ_ℓ[P_ℓ] ← 0
            /* Grow: largest |g| among inactive candidates */
            C_ℓ ← some inactive indices where M_ℓ = 0   ▷ e.g., random subset
            G_ℓ ← indices of the k largest |g_ℓ[C_ℓ]|
            M_ℓ[G_ℓ] ← 1
            Initialize θ_ℓ[G_ℓ] with small random values
        end for
    end if
end for
return θ, M
```

3. Visualization of Dynamic Sparse Training



The edge cut if the weight value is based on the weight value

4. Experimental Description(Used Dataset)

1. Used Datasets:(Opus100 SN-EN, JP-EN)

2. Tokenizer:
   Opus 100 SN-EN -> SentencePiece (BPE, vocab=8000)
   Opus 100 JP-EN -> src(JP): MeCaB,
   tgt(EN): NLTK (Same as previous)

→ **Training / Validation / Test datasets separation is same as the previou experiments**

4. Experimental Description

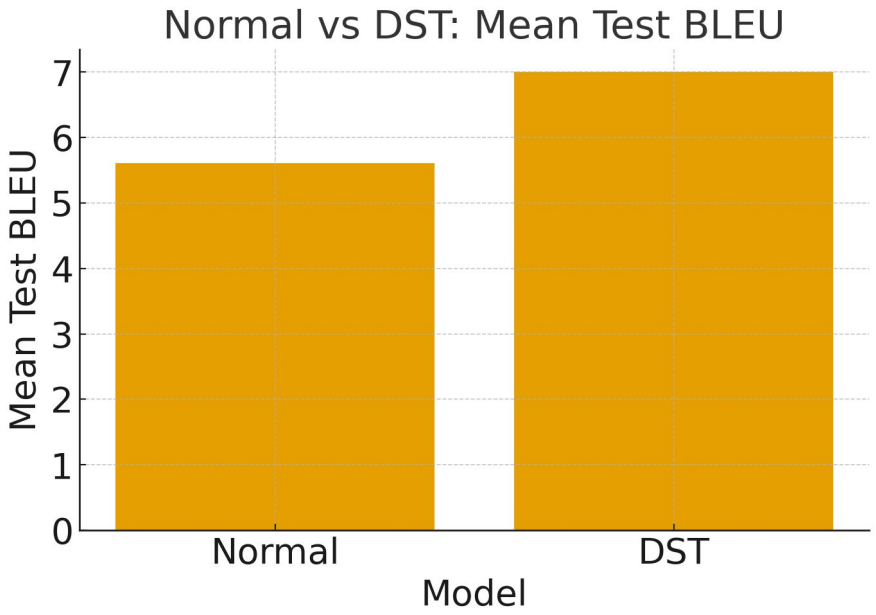
1.. FLOPs Measurement

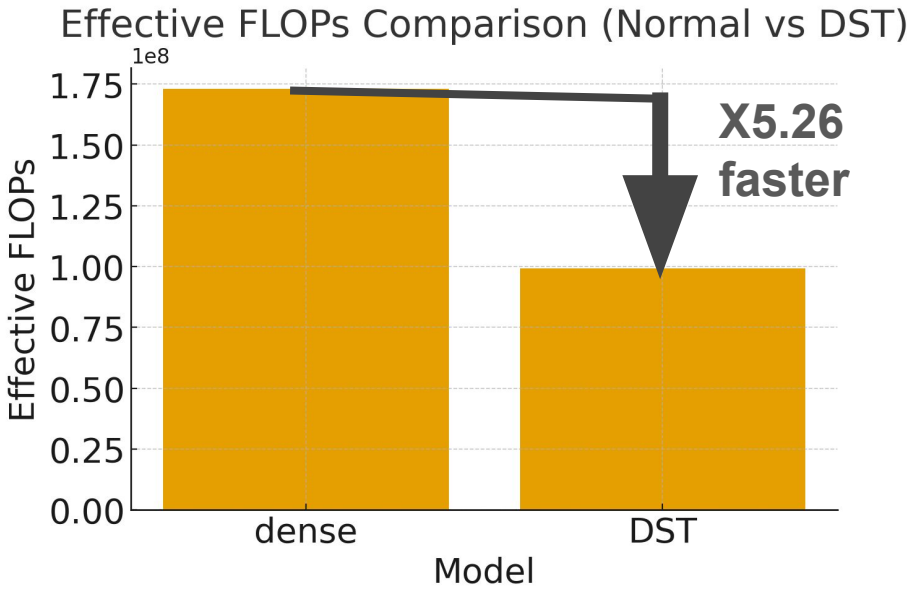
Liberary: Fvcore
Input data: 50 length dummy data
Source length = 50 tokens
Target length = 50 tokens
Head is BOS token, the other is PAD

2. BLEU score

using a **beam-search decoder** (beam size = 5)
Runnig **3** times and taking averages.

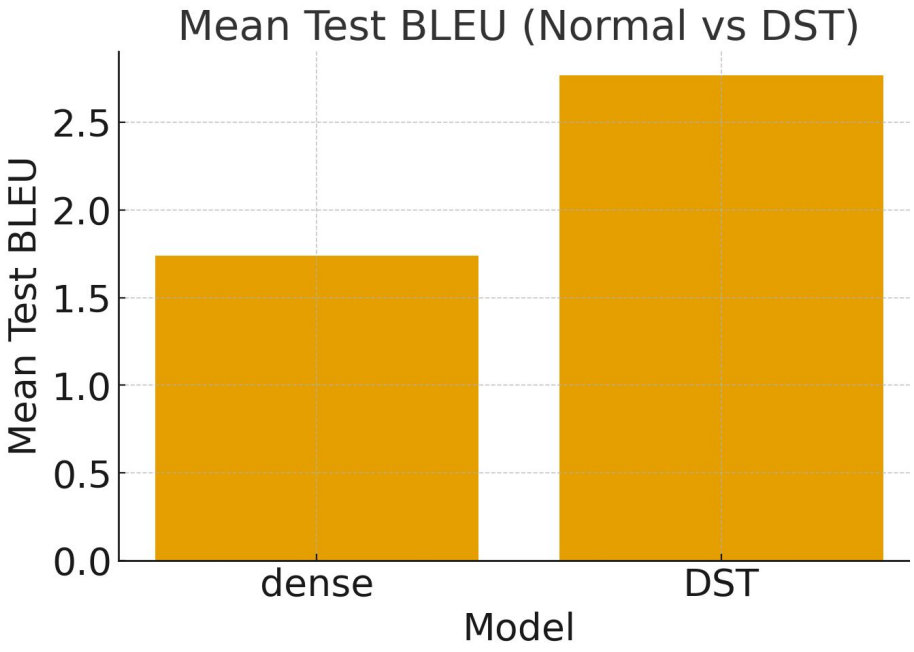
5. Results for SN-EN settings

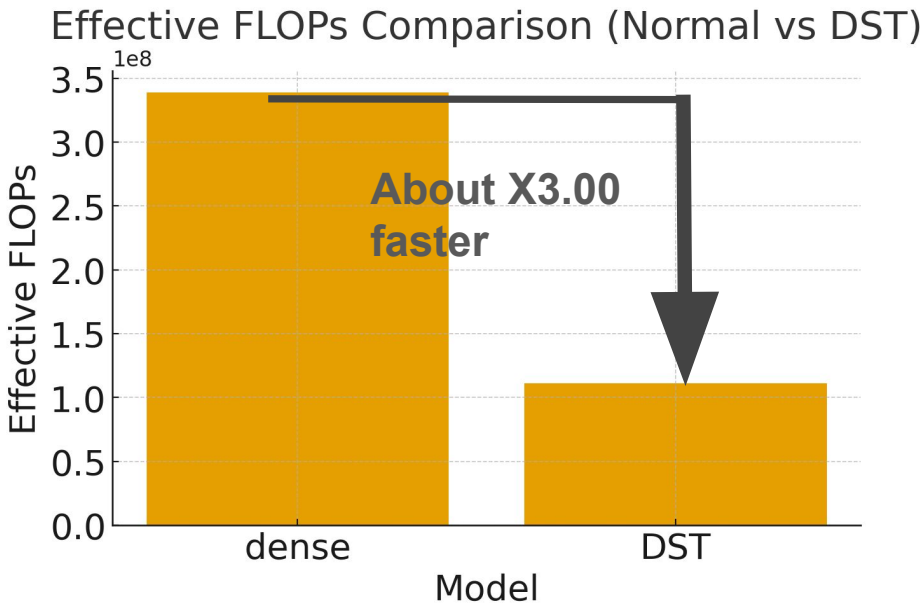




BLEU value increased compared to original one

6. Results for JP-EN settings





7. Future Works(SN->EN model and Improving Efficiency)

- **DST may effective methods as a “model compression & memory reduction” methods, but need to specific hardware that can treat the 0 value**
- **We also have to check if the model completely converges what happen to the 2 models.**

Role of each Member.

Tithira:

- Coordinating work.
- Coding: Baseline Code(from LMS) analysis and identification of room for improvement(JP->EN Transformer).
- Improving the Baseline code.
- Debugging and testing.
- Visualizing the results.
- Presentation Material Preparation.
- Presenting.

Hayafuji Fumiaki:

- Coding: Using the improved transformer and apply it to Sinhala-> English Translation.
- Further improving the model with DST.
- Visualizing the results.
- Presentation Material Preparation.
- Presenting.

Thank you!

Feel free to ask any questions.....